

PrintDeck

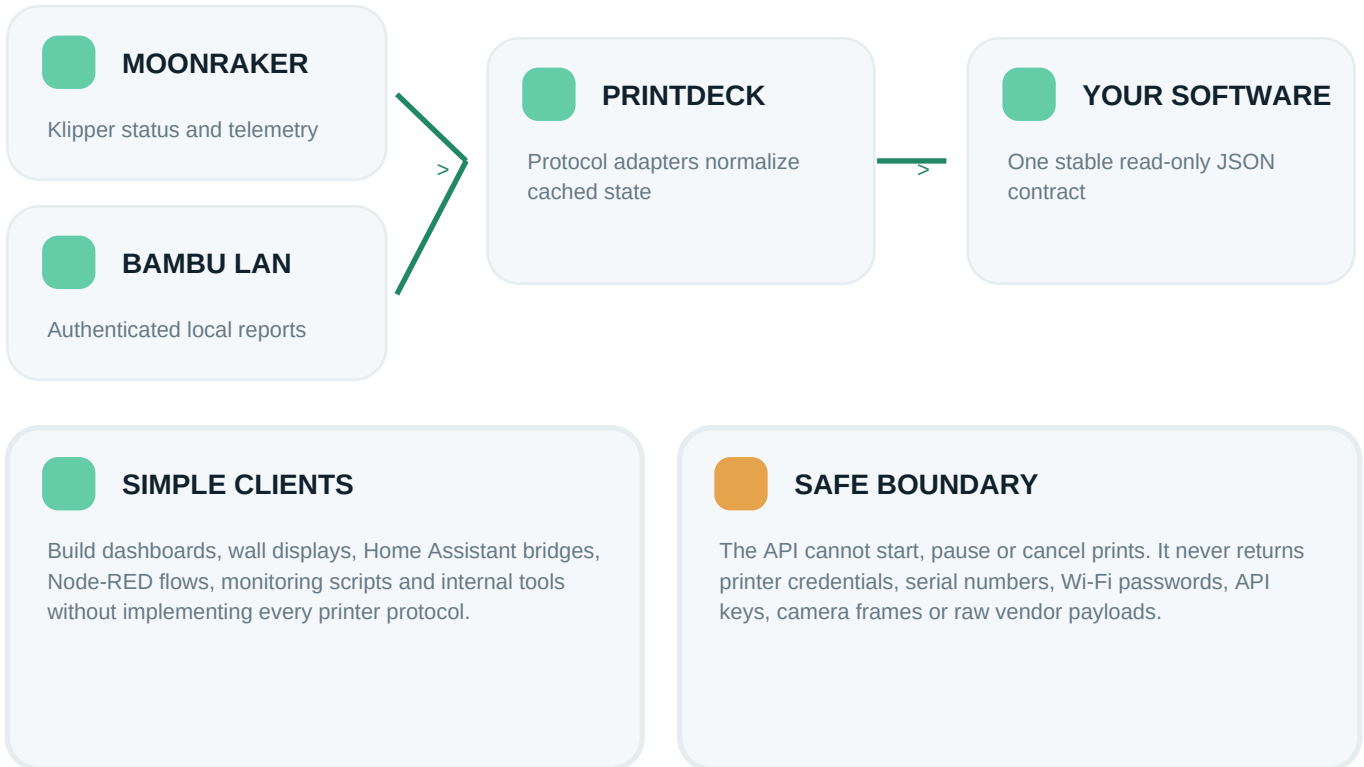
UNIFIED PRINTER API

One local, read-only JSON format for every printer



One API between your software and every printer

Printer vendors and local print servers expose different protocols, field names and status models. PrintDeck already translates those inputs for its screens. The Unified Printer API exposes that normalized, cached state as versioned JSON.



DATA PATH

printer formats -> PrintDeck normalized cache -> HTTP / MQTT -> JSON

HTTP snapshots, MQTT telemetry and observed print events share the same normalized data. No webhooks, cloud account, remote relay or print-control endpoints.

In this edition: HTTP reference (pages 3-8), MQTT topics (9), Home Assistant and Discovery (10), event semantics (11), journal JSON (12), source routing and automation (13). Event support requires compatible firmware; Home Assistant support is included in PrintDeck integration 0.1.5.

Enable access and authenticate

01

Open Web Config

Open the local address shown by PrintDeck, choose Device Settings and select Unified Printer API above Backup & Restore.

02

Enable the API

Choose Enable API. PrintDeck creates a random token and stores it locally on the device.

03

Copy the token

Use Copy token and store it as a secret in your application. Do not confuse it with a Bambu Lab LAN access code.

04

Send Bearer auth

Add the token to every request in the Authorization header. A missing or invalid token returns HTTP 401.

FIRST REQUEST

```
curl http://PRINTDECK-IP/v1/printers/status \  
-H 'Authorization: Bearer pd_your_token_here'
```

TRUSTED LAN ONLY

PrintDeck serves its local device API over HTTP. Keep it on a trusted network, do not forward the port to the internet and regenerate the token if it may have been exposed.

Endpoint reference

GET /v1/info

PrintDeck, hardware, firmware and safe network details.

GET /v1/snapshot

Device power and all printer identities and statuses in one response.

GET /v1/printers

Saved printer identities, addresses and capabilities.

GET /v1/printers/status

Compact status array for every saved printer.

GET /v1/printers/{id}

One printer identity and normalized capabilities.

GET /v1/printers/{id}/status

Connection, job, timing and temperature data.

GET /v1/printers/{id}/nozzles

Tool, material, filament and nozzle temperatures.

GET /v1/printers/{id}/materials

AMS, AMS Lite and external spool data when known.

Prefer /v1/snapshot when polling multiple printers. All routes are read-only; IDs remain stable until a profile is deleted and recreated.

Unavailable fields are null. Known offline printers still return HTTP 200 with `stale: true`. Unknown printer IDs return HTTP 404.

PrintDeck and printer identity

DEVICE INFORMATION

```
GET /v1/info
{
  "api_version": "v1",
  "product": "PrintDeck",
  "device_id": "printdeck-a1b2c3d4e5f6",
  "firmware_version": "1.0.3",
  "hardware": "amoled_1_75",
  "network": {
    "wifi_name": "Workshop Wi-Fi",
    "ipv4": "192.168.1.42",
    "hostname": "printdeck.local"
  },
  "read_only": true
}
```

PRINTER LIST

```
GET /v1/printers
{
  "api_version": "v1",
  "printers": [{
    "id": 1,
    "name": "Workshop Voron",
    "protocol": "moonraker",
    "manufacturer": "Voron",
    "model": "2.4",
    "endpoint": "192.168.1.50:7125",
    "network": {"address": "printer.local", "port": 7125},
    "selected": true,
    "reachability": "online",
    "capabilities": {"status": true, "nozzles": true, "materials": true}
  }]
}
```

Examples show selected fields. Display network.address and network.port as supplied; do not infer ports from protocol or connect directly to a printer. Older firmware may omit network.

Status and freshness

SELECTED PRINTER - FULL DETAIL

```
GET /v1/printers/1/status
{
  "api_version": "v1",
  "status": {
    "printer_id": 1,
    "connection": {
      "state": "online",
      "reachability": "online",
      "detail_level": "full",
      "stale": false,
      "updated_at_ms": 418520
    },
    "job": {
      "phase": "printing", "kind": "print",
      "condition": "busy",
      "activity": "printing", "name": "Bracket",
      "progress_percent": 42.5,
      "elapsed_seconds": 1260, "remaining_seconds": 1740,
      "current_layer": 48, "total_layers": 112
    },
    "temperatures": {
      "nozzle_current_c": 200.4, "nozzle_target_c": 200,
      "bed_current_c": 59.8, "bed_target_c": 60,
      "chamber_current_c": null
    }
  }
}
```



DETAIL LEVEL

full means the selected printer has live normalized telemetry. summary means PrintDeck has only the bounded printer-list probe.



STALE DATA

Always inspect stale and connection.state. Cached data can remain useful while a printer reconnects or is offline.

Nozzles and materials

TOOLHEADS

```
GET /v1/printers/1/nozzles
{
  "printer_id": 1, "detail_level": "full", "stale": false,
  "nozzles": [{
    "id": "T0", "active": true, "state": "ready",
    "diameter_mm": 0.4,
    "temperature": {"current_c": 133, "target_c": 200},
    "material": {"type": "PLA", "color": "#111111FF"},
    "filament_detected": true
  }]
}
```

MATERIAL SYSTEM

```
GET /v1/printers/2/materials
{
  "printer_id": 2, "available": true,
  "system": "ams_or_ams_lite", "stale": false,
  "slots": [{
    "id": "slot-0", "installed": true, "feeding": true,
    "material": "PLA", "color": "#E6B422FF",
    "remaining_percent": 72,
    "source_unit": 0, "source_slot": 0
  }],
  "external_spools": []
}
```

Array order follows the normalized tool or feeder order reported by PrintDeck. Empty arrays are valid and do not indicate an API error.

Errors, integration rules and privacy



HTTP 401

The Authorization header is missing or the Bearer token is invalid. Copy the current token from Web Config.



HTTP 403

The Unified Printer API is disabled. Enable it locally in Web Config before retrying.



HTTP 404

The route is unknown or the printer ID no longer exists. Refresh GET /v1/printers.



HTTP 429

The client exceeded one authorized request per second. Wait for Retry-After: 1 before retrying.

Client checklist

- Keep the token outside source control and logs.
- Use the printer ID as the stable key, not its display name.
- Handle null fields and empty arrays without inventing values.
- Read connection.state, detail_level and stale before telemetry.
- Send at most one request per second; faster requests return HTTP 429.
- Do not expose the PrintDeck port or token outside the trusted local network.

RATE-LIMIT RESPONSE

```
HTTP/1.1 429 Too Many Requests
Retry-After: 1
{"error":{"code":"rate_limited","message":"Unified Printer API allows one request per second."}}
```

Connect PrintDeck to your broker

Configure MQTT in Web Config > Device Settings > Unified API. PrintDeck is a client of an external broker. HTTP and MQTT can operate independently. Broker credentials are separate from the HTTP bearer token. TLS validates the certificate and hostname; a custom CA can support private certificates.

One namespace per PrintDeck

The root is `printdeck/<printdeck_id>/v1`. The table below lists suffixes relative to that root. Profile IDs survive renaming and address changes; deleting and recreating a profile creates a new identity.

info	Retained	Device identity and Discovery ownership
printers	Retained	Complete current printer catalog
availability	Retained	online/offline and the device last will
device	Live	Device power in a v1 JSON wrapper
printers/<id>/info	Live	Printer identity and capabilities
printers/<id>/status	Live	Normalized status and event journal
printers/<id>/nozzles	Live	Tool and filament telemetry
printers/<id>/materials	Live	Read-only material-system telemetry
printers/<id>/events	Live	Individual print event, QoS 0

Live messages are not retained. Broker connectivity does not prove a printer is online: inspect `connection.state`, `stale` and `detail_level`. Only the selected printer has a full live connection; inactive profiles keep their configured summary cadence. Missing values remain null.

Catalog, device and status wrappers include `_mqtt_generation`: eight hexadecimal session digits, a hyphen, then eight catalog-counter digits. Clear live caches when it changes and accept only matching reports. Refresh on Home Assistant birth; do not combine a retained catalog with telemetry from an older generation.

Home Assistant and MQTT Discovery

Choose one integration path for each PrintDeck

HACS HTTP: install PrintDeck, enable its Unified Printer API and enter the device address and API token. HACS MQTT: configure Home Assistant's MQTT integration and PrintDeck against the same broker, then select MQTT in PrintDeck's HA setup and enter the full topic root. Keep automatic Discovery disabled for either HACS path.

Native MQTT Discovery

HACS is optional for this path. Enable MQTT and automatic Home Assistant Discovery on PrintDeck. The built-in MQTT integration creates sensors and a Print events entity for each printer. Discovery configuration is retained; live telemetry and event announcements are not.

Existing sensors remain available

Event support adds a Print events entity and a Printer condition sensor. Existing sensor unique IDs stay unchanged. Known progress and elapsed time are also available from summary polling. A Print events entity stays unavailable on older firmware while ordinary sensors continue working.

HACS state and recovery

HACS MQTT waits for fresh status for the complete catalog and fresh device power before restoring aggregate sensor state. Telemetry expires after 90 seconds. Once setup is complete, a valid print event can be delivered without waiting for unrelated profiles or power telemetry. Offline, reconnect and catalog changes clear or re-baseline live state.

Switch transports and clean up Discovery

Use Reconfigure to switch HTTP/MQTT within the existing HACS entry and preserve entity identity. Moving between HACS and native MQTT Discovery changes the owning HA integration and requires updating automations. Disable Discovery while the old broker is reachable and wait for cleanup before enabling HACS. Remove obsolete retained configuration manually if that broker is no longer reachable.

Keep sources independent

Choose HACS or native Discovery per PrintDeck, even when several PrintDecks observe the same printer. HACS blocks conflicting native MQTT ownership and shows a repair notice. This prevents duplicate integration paths for one device; it does not merge observations from different PrintDecks.

Print events: what a transition means

Observe normalized state

PrintDeck detects transitions when its Unified API view is sampled. HTTP polling and the approximately five-second MQTT publishing cycle may miss intermediate states. Inactive printers keep their list-refresh cadence. Event tracking does not open additional full printer connections.

Start, pause and resume

started includes preparation. Preparing-to-printing does not produce a second start; paused-to-printing produces resumed. paused reports an observed pause. PrintDeck assigns its own local `job_id` using an observation stream and session counter, without relying on printer-specific job identifiers, names or models.

Completion, failure and cancellation

completed, failed and cancelled require a terminal phase observed after an active phase. Reaching 100 percent alone is not completion. `job_kind` distinguishes print from calibration; the supplied event blueprint excludes calibration jobs.

Progress milestones

milestone reports 25, 50 or 75. A jump across several known thresholds emits them in order, once per observed session. Progress corrections and pauses do not rearm thresholds. Unknown progress is not zero: the next known value establishes a quiet baseline. `progress_percent` is the measured value, separate from the milestone.

Attention and condition

attention marks entry into attention/error or a change between these conditions. `attention_cleared` requires a known non-attention condition. This does not imply that the print failed. condition values are unknown, normal, ready, busy, attention and error; raw vendor warning payloads are not included.

Quiet startup and bounded history

An already-running print does not emit started on first observation or recovery. A restart or endpoint/protocol change creates a new stream. A gap in reliable observations creates a quiet baseline and can assign a new local job session. Each profile keeps at most eight records in RAM. This is an observation journal, not durable print history or guaranteed delivery.

Event journal in HTTP and MQTT status

Every status resource can include the optional `print_events` object. It is also inside each printer status in `/v1/snapshot` and in MQTT printers `/<id>/status`. This example shows that object after the first observed start.

OPTIONAL STATUS.PRINT_EVENTS OBJECT

```
"print_events": {
  "stream_id": "0123456789abcdef",
  "sequence": 1,
  "job_id": "0123456789abcdef-1",
  "observed_at_ms": 124000,
  "events": [{
    "stream_id": "0123456789abcdef",
    "sequence": 1,
    "job_id": "0123456789abcdef-1",
    "observed_at_ms": 123000,
    "event_type": "started",
    "job_kind": "print",
    "progress_percent": 0,
    "milestone": 0,
    "condition": "busy"
  }]
}
```

`stream_id` is 16 lowercase hexadecimal characters. `sequence` increases per stream, and events are ordered by sequence. An empty history starts at sequence 0. `job_id` appends a local session counter and may be null when no session exists. IDs are scoped to an observation source, not globally shared printer history.

`observed_at_ms` uses PrintDeck's monotonic clock, not Unix time. The outer time describes the observation view; record times describe detected events. Compare these times only within the same device runtime. `progress_percent` is a number from 0 to 100 or null; `milestone` is 25/50/75 only for milestone events and 0 otherwise.

Consumer rule: quietly seed a cursor for each PrintDeck/profile/stream, then accept only increasing sequences. HACS ignores records older than 30 seconds and re-baselines after transport reconnect or a new stream. Repeated status messages do not repeat automations. Polling outages or journal eviction can lose events.

Source routing and automation

Each source is a saved printer profile on one PrintDeck. Three PrintDecks observing the same physical printer intentionally produce three independent event streams. A routing key includes the PrintDeck ID so automations can distinguish them.

THREE DIFFERENT SOURCES, EVEN FOR ONE PHYSICAL PRINTER

```
routing_key = <printdeck_id>:<printer_id>  
event_id = <printdeck_id>:<printer_id>:<stream_id>:<sequence>
```

```
printdeck-a1b2c3d4e5f6:1  
printdeck-112233445566:1  
printdeck-ffeeddcbbaa:1
```

Individual MQTT events

Topic: `printdeck/<printdeck_id>/v1/printers/<printer_id>/events`. Each JSON message contains one event record from page 12 plus `printdeck_id`, `printer_id` as a string, `routing_key` and `event_id`. Delivery is non-retained QoS 0. Reconnect does not replay old announcements. Native Discovery event entities consume this topic; a missed message can lose an event.

HACS event entity and event bus

HACS reads the status journal over HTTP or MQTT. It updates the printer's Print events entity and emits `printdeck_event` on the Home Assistant event bus. The payload contains the event fields, source IDs, `routing_key`, `event_id`, `printer_name` and `device_id`. Here `device_id` is the HA printer-device registry ID; `printdeck_id` identifies the physical PrintDeck.

Configure one automation for current and future printers

Use the Respond to print events blueprint from PrintDeck/home-assistant, under `blueprints/automation/printdeck/print_events.yaml`. Choose event types and actions. Leave PrintDeck ID empty for all sources, or enter a `printdeck_id` to select one. Variants are provided in Polish, Spanish, French, German and Simplified Chinese. The blueprint requires HACS; native MQTT event entities use HA's standard entity automations.

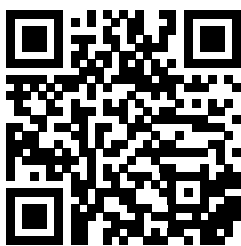
Test the complete path

After installing PrintDeck integration 0.1.5 or later, restart Home Assistant. In Developer Tools > Events, listen to `printdeck_event` and observe a real print transition. Check `event_type`, `job_id` and `routing_key`. A sensor refresh or an already-running job at startup should not be interpreted as a new started event.

PrintDeck

ONE FORMAT. MANY PRINTERS.

Build local software against the data model PrintDeck already uses for its own screens.



DEVELOPER DOCUMENTATION

Examples, use cases and current guide

Scan to open the Unified Printer API page and download the latest documentation.

<https://printdeck.xyz/unified-printer-api/>